

MATH00041: Introduction to Programming

Data Types – Variables – Names – Assignments

Lecture 1

Archontis Giannakidis, PhD

Course Logistics (1/4): Instructor Details

Archontis Giannakidis, PhD

Email: archontis.giannakidis@ntu.ac.uk

Tel: 0115-84-83968

Office: Mary Ann Evans, Room 302, Clifton Campus

Course Logistics (2/4): Weekly Sessions

- 2-hour lecture
- 2-hour seminar
- MATH00041 learning room @ NOW

Course Logistics (3/4): Assessment

- Class Test (Week 6, 30%)
- Coursework (Towards the end, 70%)

Course Logistics (4/4): Course Outline

Week – 1 : Data types, variables, names, assignments.

Week – 2 : Text strings and Numbers (Booleans, integers, floats, complex)

Week – 3 : Tuples and lists and arrays

Week – 4 : Choose with if and Loops (while, for)

Week – 5 : Dictionaries and sets

Week – 6 : Functions and Importing Modules & Packages

Week – 7 : Objects & Classes – Inheritance

Week – 8 : numpy library I

Week – 9 : matplotlib

Week – 10 : numpy library II

Week – 11 : pandas library and statistics, scipy.stats and seaborn library

Outline of this lecture

- **Syntax**
- **Operators**
- **Data types, variables, names, assignments**

Python Syntax

Execute Python Syntax

Python syntax can be executed by writing directly in the Command Line:

```
>>> print("Hello, World!")  
Hello, World!
```

Python Indentation

Indentation refers to the spaces at the beginning of a code line.

Whereas in other programming languages, the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

```
>>> if 5 > 2:
...     print("Five is greater than two!")
...
Five is greater than two!
>>>
```

```
>>> if 5 > 2:
... print("Five is greater than two!")
...   File "<stdin>", line 2
...     print("Five is greater than two!")
...     ^
IndentationError: expected an indented block
>>>
```

Comments

Python has commenting capability for the purpose of in-code documentation.

Comments start with a #, and Python will render the rest of the line as a comment:

Example:

Comments in Python:

#This is a comment.

```
print("Hello, World!")
```

Operators

Assignment operators are used in programming languages to assign values to variables. They are essential for initialising variables and updating their values during the execution of the program:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Python Arithmetic Operators

Arithmetic operators are symbols used with numeric values to perform various mathematical operations such as addition, subtraction, multiplication, and division.

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Python Assignment Operators

Assignment operators are used in programming languages to assign values to variables. They are essential for initialising variables and updating their values during the execution of the program:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Python Comparison Operators

Comparison operators are used to compare two values. They help to determine whether the values are equal, greater than, less than, or not equal to each other. These operators are commonly used in programming languages to make comparisons and decisions based on the results:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Python Logical Operators

Conditional statements can be combined using logical operators. These operators help in creating more complex conditions for decision-making in a program:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

Python Identity Operators

Identity operators compare objects by their memory location, not just their values.

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Python Membership Operators

Membership operators are used to test whether a sequence exists within an object.

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Python Bitwise Operators

Bitwise operators are used to compare binary numbers. These operators perform operations on individual bits of the numbers. They are useful for performing low-level operations such as bitwise arithmetic, bit-manipulation, and setting or clearing of bits.

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	x & y
	OR	Sets each bit to 1 if one of two bits is 1	x y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

Operator Precedence

Operator precedence is a fundamental concept in programming that defines the order in which operations are executed. This order is essential to ensure the correctness and accuracy of the program's output. The table below illustrates the precedence order, with the highest priority at the top of the list. Understanding operator precedence is crucial for efficient and error-free coding:

Operator	Description
()	Parentheses
**	Exponentiation
+x -x ~x	Unary plus, unary minus, and bitwise NOT
* / // %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR

Variables, names, assignments, data types

Python Variables

Variables are containers for storing data values.

Creating Variables

In Python, unlike many other programming languages, you don't need to declare a variable before using it. When you assign a value to a variable for the first time, Python automatically creates the variable. This means that the variable's data type is determined by the type of value assigned to it. For example, if you assign a string value to a variable, the variable will be of type string. Similarly, if you assign a numeric value to a variable, the variable will be of type integer or float depending on the value assigned. It's important to keep in mind that if you try to use a variable before assigning a value to it, Python will raise an error.

Example:

```
x = 5  
y = "John"  
print(x)  
print(y)
```

In programming, variables store values and can be used throughout the program. In languages like Python, variables don't need a data type declaration. Instead, the type is inferred from the value assigned and can be changed later for flexibility.

Example:

```
x = 4          # x is of type int
x = "Sally"    # x is now of type str
print(x)
```

Casting

If you want to specify the data type of a variable, this can be done with casting. About this topic we will discuss it in more detail later on.

Example

```
x = str(3)    # x will be '3'  
y = int(3)    # y will be 3  
z = float(3)  # z will be 3.0
```

Get the Type

In programming, a variable is a container that stores a value. Each variable has a specific data type, which refers to the kind of value it stores, such as a number, string, or boolean. To determine the data type of a variable in your code, you can use the `type()` function. This function takes the variable as an argument and returns the data type of the value it stores.

Example:

```
x = 5  
y = "John"  
print(type(x))  
print(type(y))
```

Single or Double Quotes?

String variables can be declared either by using single or double quotes:

Example:

```
x = "John"  
# is the same as  
x = 'John'
```

Case-Sensitive

Variable names are case-sensitive.

Example:

```
a = 4  
A = "Sally"  
#A will not overwrite a
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for Python variables:

- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Example:

Legal variable names:

```
myvar = "John"  
my_var = "John"  
_my_var = "John"  
myVar = "John"  
MYVAR = "John"  
myvar2 = "John"
```

Example:

Illegal variable names:

```
2myvar = "John"  
my-var = "John"  
my var = "John"
```

Multi Words Variable Names

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

- Camel Case

Each word, except the first, starts with a capital letter:

```
myVariableName = "John"
```

- Pascal Case

Each word starts with a capital letter:

```
MyVariableName = "John"
```

- Snake Case

An underscore character separates each word:

```
my_variable_name = "John"
```

Many Values to Multiple Variables

In Python, you can assign values to multiple variables at once, all on a single line:

Example:

```
x, y, z = "Orange", "Banana", "Cherry"  
print(x)  
print(y)  
print(z)
```

```
output: Orange  
output: Banana  
output: Cherry
```

One Value to Multiple Variables

You can assign a single value to multiple variables in a single line of code:

Example:

```
x = y = z = "Orange"  
print(x)  
print(y)  
print(z)
```

```
output: Orange  
output: Orange  
output: Orange
```

Unpack a Collection

Python provides a convenient way to extract values from a list or tuple and assign them to individual variables. This process is called unpacking, and it can save you time and effort when dealing with complex data structures. To perform unpacking, simply use the assignment operator with the list or tuple on the right-hand side and the variables on the left-hand side. This can be especially useful when working with long lists or tuples as it can help reduce the amount of code you need to write and make your code more concise.

Example:

Unpack a list:

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
print(z)
```

output: apple

output: banana

output: cherry

Output Variables

The Python `print()` function is often used to output variables.

Example:

```
x = "Python is awesome"  
print(x)
```

output: Python is awesome

In the `print()` function, you output multiple variables separated by a comma:

Example:

```
x = "Python"  
y = "is"  
z = "awesome"  
print(x, y, z)
```

output: Python is awesome

For numbers, the + character works as a mathematical operator

Example:

```
x = 5
y = 10
print(x + y)
```

output: 15

In the print() function, when you try to combine a string and a number with the + operator, Python will give you an error:

```
>>> x = 5
>>> y = "John"
>>> print(x + y)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>>
```

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types:

```
>>> x = 5
>>> y = "John"
>>> print(x, y)
5 John
```

Global Variables

Variables created outside of a function (as in the examples above) are known as global variables. Everyone can use global variables, both inside of functions and outside.

Example:

Create a variable outside of a function and use it inside the function.

```
x = "awesome"
```

```
def myfunc():  
    print("Python is " + x)
```

```
myfunc()
```

output: Python is awesome

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

Example:

Create a variable inside a function with the same name as the global variable.

```
x = "awesome"
```

```
def myfunc():  
    x = "fantastic"  
    print("Python is " + x)
```

```
myfunc()
```

```
print("Python is " + x)
```

output: Python is fantastic

output: Python is awesome

The global Keyword

Typically, when you create a variable inside a function, that variable is local and can only be used inside that function.

- You can use the global keyword to create a global variable inside a function.
- Also, use the global keyword if you want to change a global variable inside a function.

Example:

If you use the global keyword, the variable belongs to the global scope:

```
def myfunc():  
    global x  
    x = "fantastic"  
  
myfunc()  
  
print("Python is " + x)
```

output: Python is fantastic

Example:

To change the value of a global variable inside a function, refer to the variable by using the global keyword:

```
x = "awesome"  
def myfunc():  
    global x  
    x = "fantastic"  
  
myfunc()  
  
print("Python is " + x)
```

output: Python is fantastic

Python Data Types

In programming, data type is an important concept.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

- | | |
|--------------------|------------------------------|
| 1. Text Type: | str |
| 2. Numeric Types: | int, float, complex |
| 3. Sequence Types: | list, tuple, range |
| 4. Mapping Type: | dict |
| 5. Set Types: | set, frozenset |
| 6. Boolean Type: | bool |
| 7. Binary Types: | bytes, bytearray, memoryview |
| 8. None Type: | NoneType |

Getting the Data Type

You can get the data type of any object by using the `type()` function:

Example:

Print the data type of the variable x:

```
x = 5  
print(type(x))
```

output: <class 'int'>

Setting the Data Type

In Python, the data type is set when you assign a value to a variable:

Example	Data Type
x = "Hello World"	str
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range
x = {"name": "John", "age": 36}	dict
x = {"apple", "banana", "cherry"}	set
x = frozenset({"apple", "banana", "cherry"})	frozenset
x = True	bool
x = b"Hello"	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview
x = None	NoneType

Setting the Specific Data Type

If you want to specify the data type, you can use the following constructor functions:

Example	Data Type
<code>x = str("Hello World")</code>	<code>str</code>
<code>x = int(20)</code>	<code>int</code>
<code>x = float(20.5)</code>	<code>float</code>
<code>x = complex(1j)</code>	<code>complex</code>
<code>x = list(("apple", "banana", "cherry"))</code>	<code>list</code>
<code>x = tuple(("apple", "banana", "cherry"))</code>	<code>tuple</code>
<code>x = range(6)</code>	<code>range</code>
<code>x = dict(name="John", age=36)</code>	<code>dict</code>
<code>x = set(("apple", "banana", "cherry"))</code>	<code>set</code>
<code>x = frozenset(("apple", "banana", "cherry"))</code>	<code>frozenset</code>
<code>x = bool(5)</code>	<code>bool</code>
<code>x = bytes(5)</code>	<code>bytes</code>
<code>x = bytearray(5)</code>	<code>bytearray</code>
<code>x = memoryview(bytes(5))</code>	<code>memoryview</code>

Python Numbers

There are three numeric types in Python:

1. int
2. float
3. complex

When you assign a value to a variable, it creates a variable of numeric data type. This means that the variable can hold numerical values, such as integers, decimals, or fractions. Once a numeric variable is created, you can perform its arithmetic operations, such as addition, subtraction, multiplication, and division. Numeric variables are used extensively in programming for a wide range of applications, from simple calculations to complex mathematical algorithms:

Example:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```

To determine the type of an object in Python, make use of the `type()` function:

Example:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```

```
print(type(x))
print(type(y))
print(type(z))
```

output: <class 'int'>

output: <class 'float'>

output: <class 'complex'>

1. Integer

An integer or int is a mathematical concept that refers to a number with no fractional component, indicating a complete whole unit. It can be either positive or negative and has an infinite length.

Example:

Integers:

```
x = 1
y = 35656222554887711
z = -3255522
```

```
print(type(x))
print(type(y))
print(type(z))
```

output: <class 'int'>

output: <class 'int'>

output: <class 'int'>

2. Float

A float is a numerical value that includes one or more decimals, and it can be either positive or negative.

Example:

Floats:

```
x = 1.10
```

```
y = 1.0
```

```
z = -35.59
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

```
output: <class 'float'>
```

```
output: <class 'float'>
```

```
output: <class 'float'>
```

Scientific numbers in float can be represented with "e" for power of 10

Example:

Floats:

```
x = 35e3
```

```
y = 12E4
```

```
z = -87.7e100
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

```
output: <class 'float'>
```

```
output: <class 'float'>
```

```
output: <class 'float'>
```

3. Complex

Complex numbers are written with a "j" as the imaginary part:

Example:

Complex:

```
x = 3+5j
```

```
y = 5j
```

```
z = -5j
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

```
output: <class 'complex'>
```

```
output: <class 'complex'>
```

```
output: <class 'complex'>
```

Type Conversion

You can convert from one type to another with the `int()`, `float()`, and `complex()` methods:

Example:

Convert from one type to another:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```

```
#convert from int to float:
```

```
a = float(x)
```

```
#convert from float to int:
```

```
b = int(y)
```

```
#convert from int to complex:
```

```
c = complex(x)
```

```
print(a)
```

```
output: 1.0
```

```
print(b)
```

```
output: 2
```

```
print(c)
```

```
output: (1+0j)
```

```
print(type(a))
```

```
output: <class 'float'>
```

```
print(type(b))
```

```
output: <class 'int'>
```

```
print(type(c))
```

```
output: <class 'complex'>
```

Python Casting

Specify a Variable Type

There may be times when you want to specify a type on to a variable. This can be done with casting. Python is an object-orientated language, and as such, it uses classes to define data types, including its primitive types.

Casting in Python is therefore done using constructor functions:

- `int()` - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)
- `float()` - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- `str()` - constructs a string from a wide variety of data types, including strings, integer literals and float literals

Example:

Integers:

```
x = int(1)    # x will be 1
y = int(2.8)  # y will be 2
z = int("3")  # z will be 3
```

Floats:

```
x = float(1)    # x will be 1.0
y = float(2.8)  # y will be 2.8
z = float("3")  # z will be 3.0
w = float("4.2") # w will be 4.2
```

Strings:

```
x = str("s1")  # x will be 's1'
y = str(2)     # y will be '2'
z = str(3.0)   # z will be '3.0'
```

Thank you